

D1780R0

Modular Relaxed Dependencies: A new approach to the Out-Of-Thin-Air Problem

Draft Proposal, 2019-06-14

This version:

<http://wg21.link/D1780R0>

Authors:

[Mark Batty](#) (University of Kent)

[Simon Cooksey](#) (University of Kent)

[Scott Owens](#) (University of Kent)

[Anouk Paradis](#) (Ecole Polytechnique)

[Marco Paviotti](#) (University of Kent)

[Daniel Wright](#) (University of Kent)

Audience:

SG1

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

<https://github.com/MBattyResearch/compositional-eventstructures/c++-pxxx/>

Abstract

This note describes Modular Relaxed Dependency (MRD), an alteration of the existing concurrency specification that allows compiler optimisations while forbidding thin-air values. It has two key advantages over previous suggestions: first, it is a relatively contained change that leaves much of the existing concurrency definition unaltered; second, and in contrast to other solutions, the cost of evaluating the semantics is relatively low. We present the context, the ideas behind the change, and a tool that executes the amended model.

Table of Contents

- 1 **Recap**
- 2 **Our Solution**
- 3 **Modular Relaxed Dependency**
- 4 **MRDer**
- 5 **Conclusion**

§ 1. Recap

From its introduction in 2011, the C++ concurrency model has faced a great deal of scrutiny, leading to various refinements of the standard, and a list of fixes to known issues that are yet to be incorporated. [P0668R5: Revising the C++ Memory Model](#) describes changes needed to incorporate the fixes of [RC11](#), the latest revision of the existing C++ concurrency design, solving most known problems.

This process of scrutiny and refinement has produced a mature model, but it has also uncovered major problems. The out-of-thin-air problem, discussed most recently in [P1217R1: Out-of-thin-air, revisited, again](#) does not yet have a convenient solution: existing solutions -- like the [Promising Semantics](#) -- discard the mature concurrency model of the standard and start afresh with a different paradigm. Implementing such a change would require a wide-ranging rewrite of the standard.

The out-of-thin-air problem arises from an under-specification of which program dependencies ought to order memory accesses. The trouble is neatly explained by example.

LB

```
// Thread 1
r1 = x.load(memory_order::relaxed);
y.store(1, memory_order::relaxed);

// Thread 2
r2 = y.load(memory_order::relaxed);
x.store(1, memory_order::relaxed);
```

In the example above, called Load Buffering (LB), C++ allows the execution where `r1` and `r2` have value 1 at the end of execution. This outcome must be allowed to permit efficient compilation of relaxed accesses to plain loads and stores on the Power and ARM architectures, where the corresponding behaviour is allowed.

LB+datas (P1217R1 OOTA Example 1, JCTC4)

```
// Thread 1
r1 = x.load(memory_order::relaxed);
y.store(r1, memory_order::relaxed);

// Thread 2
r2 = y.load(memory_order::relaxed);
x.store(r1, memory_order::relaxed);
```

In the example above, taken from [P1217R1](#), there are now data dependencies from load to store and no explicit write of any value. Surprisingly, the outcome 42 is allowed by the C++ standard -- we say that the value 42 is conjured from *thin-air*.

LB+ctrls (P1217R1 OOTA Example 2, JCTC13)

```
// Thread 1
r1 = x.load(memory_order::relaxed);
if(r1 == 42) {
    y.store(42, memory_order::relaxed);
}

// Thread 2
r2 = y.load(memory_order::relaxed);
if(r2 == 42) {
    x.store(42, memory_order::relaxed);
}
```

A similar behaviour can be exhibited using control dependencies, as above. The thin-air outcomes of LB+datas and LB+ctrls are the result of a design choice that aims to permit compiler optimisation: some optimisations remove syntactic dependencies, so if C++ bestows ordering to no dependencies, the compiler is free to remove what it likes.

LB+false+ctrl (JCTC6)

```

// Thread 1
r1 = x.load(memory_order::relaxed);
if(r1 == 42) {
    y.store(42, memory_order::relaxed);
} else {
    y.store(42, memory_order::relaxed);
}

// Thread 2
r2 = y.load(memory_order::relaxed);
if(r2 == 42) {
    x.store(42, memory_order::relaxed);
}

```

In Thread 1 above, a compiler may spot that, whatever value is read, the write will be made regardless, optimising to the program below, and allowing both threads to read 42:

```

// Thread 1
r1 = x.load(memory_order::relaxed);
y.store(42, memory_order::relaxed);

```

As it stands, the standard allows both threads to read 42 and hence it permits the optimisation, but it also allows the thin-air outcomes in LB+datas and LB+ctrls. [P1217R1](#) explains that these outcomes cannot occur in practice and must be forbidden to enable reasoning and formal verification.

§ 2. Our Solution

[P0422R0: Out-of-Thin-Air Execution is Vacuous](#) highlights a series of litmus tests, borrowed from Java, that constrain the ideal behaviour of an improved semantics (e.g. JCTC4, JCTC13 and JCTC6 above). The paper makes the observation that the thin-air execution of each test features a cycle in $(\text{rf} \cup \text{sdep})$ where *sdep*, *semantic dependency*, is an as yet undefined relation that captures only the real dependencies that the compiler will leave in place. If only we forbid cycles in $(\text{rf} \cup \text{sdep})$, then the model would have the behaviour we desire.

[P1217R1](#) highlights a simple way to guarantee a lack of thin-air behaviour: forbid the reordering of relaxed loads with following stores. This is equivalent to forbidding cycles in $(\text{rf} \cup \text{sb})$, and is sufficient to avoid cycles in $(\text{rf} \cup \text{sdep})$. This approach is controversial because it may have a substantial overhead, especially on GPUs.

Our solution is to provide a definition of **sdep**, and to forbid cycles in (**rf** $\cup$ **sdep**). We have a (sketch) proof that our solution does not incur any overhead at all; that it supports the optimum compilation strategies for Power, ARM, X86 and RISC-V; and that a DRF-SC result holds.

The standard describes the three stages of working out the concurrent behaviour of a program:

1. Generate a list of pre-executions from the source of a program, each corresponding to an arbitrary choice of read values.
2. Pair each pre-execution with reads-from and modification order relations that capture the dynamic memory behaviour of the program, and filter, keeping those that are *consistent* with the rules of the memory model.
3. Finally, check for races: race free programs have the consistent executions calculated in step 2, racy programs have undefined behaviour.

We augment this scheme by calculating **sdep** in step 1.

§ 3. Modular Relaxed Dependency

This section explains the idea behind MRD by example, referring back to the programs already discussed.

LB+data Thread 1

First we discuss LB+datas. C++ allows the program to read 42, despite there being no write of 42. It is the clearest example of thin-air behaviour and it should be forbidden.

```
r1 = x.load(memory_order::relaxed);
y.store(r1, memory_order::relaxed);
```

Above is the first thread of LB+datas. We will contrast the pre-execution that is constructed in the existing C++ definition with the structure built by MRD.

 Execution of LB thread	 Execution of LB thread	 MRD Event Structure of LB thread 1
1	1	
C++ Pre-executions		MRD Event Structure

On the left above is a list of pre-executions of this first thread -- note that we restrict the value set here to 0 and 1 to keep the diagrams tractable, so this list covers all of them. The C++ model intentionally ignores dependencies, so the events of each pre-execution are ordered only by sequenced before.

On the right is the structure generated by MRD (an *event structure* in the jargon). It is similar to the list of executions on the left, but a red zigzag links events 1 and 3. The structure that MRD generates represents all possible executions in the same graph, and the zigzag edge, called *conflict*, indicates a choice that must be made: a single execution can contain a read of value 0 at `x` or a read of value 1, not both. This structure includes *sdep* edges that allow us to forbid the unwanted outcome (we will explain their construction shortly). MRD produces pre-executions compatible with the existing C++ standard. Each pre-execution of an MRD structure is a path from top to bottom of the graph, making a choice at each conflict edge. In this program the pre-executions are {1, 2} and {3, 4}.

In LB+datas, we are interested in the outcome where both threads read a non-zero value. MRD generates one pre-execution for thread 1 where this is the case, and another similar one for thread 2. Both feature a semantic dependency from the read to the write, and a consequent cycle in (*rf* $\cup$ *sdep*). The outcome is forbidden as a result.

LB+ctrl Thread 1

```
r1 = x.load(memory_order::relaxed);
if (r1 == 42) {
    y.store(42, memory_order::relaxed);
}
```

MRD treats LB+ctrl similarly, with a small difference in the generated structure. Recall Thread 1 above.

 Execution of LB+ctrl thread 1	 Execution of LB+ctrl thread 1	 MRD Event Structure of LB+ctrl thread 1
C++ Pre-executions		MRD Event Structure

In this program, the write is only made if value 42 is read, and that is reflected in the pre-executions of C++ and also the structure generated by MRD. In LB+ctrls, both threads are of this form, and the thin-air outcome is once again forbidden by the presence of the semantic dependency.

LB+false Thread 1

```
r1 = x.load(memory_order::relaxed);
if (r1 == 1) {
    y.store(1, memory_order::relaxed);
} else {
    y.store(1, memory_order::relaxed);
}
```

The first thread of LB+false+ctrl, given above, has a false dependency from the load of `x` to the store of `y`. In hardware concurrency models, the control structure of the corresponding code would induce control dependencies, provide ordering, and forbid the outcome where both threads read 42. In C++ however, we want to allow the optimiser to hoist the store to `y` above the if-statement, so we cannot enforce this dependency.

 Execution of LB+false thread 1	 Execution of LB+false thread 1	 MRD Event Structure of LB+false thread 1
C++ Pre-executions		MRD Event Structure

C++ calculates the list of pre-executions above on the left. To allow the optimisation, C++ again omits any dependency ordering, but in this case that turns out to be appropriate.

MRD again combines all of the executions into a single structure, indicating with the red zigzag of conflict where choices must be made to identify a single execution. There is enough information in this structure to spot that regardless of the choices made, a write of value 1 to `y` is always performed. This is precisely what the machinery of MRD does as it builds the structure, and as a consequence it omits the dependency. We say that the write of `y` has been *lifted* above its dependency on the load of `x` because this mechanism is reminiscent of hoisting. Without the dependency, the execution of LB+false+ctrl where both threads read 1 is allowed, and the hoisting optimisation is sound, as intended.

LB Thread 1

We now discuss the machinery that performs lifting in MRD, using LB as an example. Recall that the relaxed outcome where both threads read 1 must be allowed, otherwise relaxed atomics cannot be implemented efficiently as plain loads and stores on the Power and ARM architectures.

```
r1 = x.load(memory_order::relaxed);
y.store(1, memory_order::relaxed);
```

We describe how MRD calculates the executions of the first thread of LB, given above.

 Execution of LB thread 1	 Execution of LB thread 1	 MRD Event Structure of LB thread 1
C++ Pre-executions		MRD Event Structure

MRD builds its structures following the program syntax. In contrast to other models, it works from back to front, so for thread 1 of LB, MRD will build the event corresponding to the store of `y` and then build the events corresponding to the load of `x`. The calculation of semantic dependencies is made by using an intermediate representation of dependency called *justification*. Justification records more

information than semantic dependency: it tracks all of the choices that lead to the execution of a particular write.

To build the structure for the first thread of LB, MRD first constructs a write event corresponding to the store to `y`. At this point, before MRD has interpreted the load, we have a structure with a single write. No choices need to be made to reach the write, so MRD builds a justification reflecting this, linking the empty set to the write. In this case, we say that the write of `y` is *independently justified*.

Next MRD interprets the load of `x`. There are two possible outcomes: either 0 is read from `x`, followed by the write of `y`, or 1 is read from `x`, followed by the write of `y`. MRD creates two read events and links them with a conflict edge to make it clear that only one can occur. The event representing the write of `y` is duplicated, with one instance following each read.

On interpreting the load of `y`, the justification relation is updated. Now there is a choice to be made before reaching the write events, we can read either 0 or 1 from `x`. In this case, that choice is of no consequence -- the same value is stored to `y` regardless. MRD therefore marks each write as independent in the justification relation.

Once MRD has interpreted the whole program, the justification relation is used to construct the `sdep` relation. In this case `sdep` is empty.

LB+data Thread 1

```
r1 = x.load(memory_order::relaxed);  
y.store(r1, memory_order::relaxed);
```

MRD builds its structure for the first thread of LB+data similarly, but here the stores are dependent on the state of the registers. MRD has a symbolic representation of the register store that it resolves as it interprets the program. On interpreting the load of `x`, the event corresponding to the store of `y` is duplicated, and its value is resolved. This leads to the creation of two events, one writing 0 to `y`, and the other writing 1 to `y`.

The conflicting reads are constructed as before, but the construction of justification is different. Now the choice of read *does* have a bearing on what write comes next. Reading 0 from `x` will result in a write of 0 to `y`, so we say that the read of 0 from `x` *justifies* the write of 0 to `y`. Similarly for loading 1 from `x`, we say that the read of 1 from `x` *justifies* the write of 1 to `y`. MRD translates these justifications into dependencies from the reads to the writes.



MRD Event Structure of LB thread 1

Through these steps, MRD produces the structure above, and forbids the outcome where both threads read 1.

§ 4. MRDer

We have built a tool that takes C-like programs as input and evaluates them under the C++ memory model augmented with MRD (MRDC). The language supports non-atomics, atomics, RMWs, fences, forks and joins. This combination uses the RC11 model, incorporating all known fixes to the original C++ specification. Most litmus tests are evaluated in under a minute on a modestly spec'd computer (16GB of Memory, Intel i5-5250U @ 1.60GHz (2.5GHz turbo)).

The performance of the tool is largely proportional to the size of the program being evaluated. For example, a variant of ISA2 with fences is evaluated in 1.5s, whereas without the fences it is evaluated in 0.8s. We have run the tool on 100 hand-chosen tests, and we believe the performance to be compatible with automated testing.

We will ultimately release the code but for now, in development, please eagerly email us with questions about the tool. We can execute tests for you, or we can work together to get MRDer working in conjunction with your own tools.

The RFUB test, below, was recently discussed in [P1217R1](#).

```
// Thread 1:
r1 = x.load(memory_order::relaxed);
y.store(r1, memory_order::relaxed);

// Thread 2:

bool assigned_42(false);
r2 = y.load(memory_order::relaxed);
if(r2 != 42) {
    assigned_42 = true;
    r2 = 42;
}
x.store(r2, memory_order::relaxed);
assert_not(assigned_42);
```

We used MRDer to evaluate this test, and the outcome in question is allowed by MRDC as desired: `r1 == 42 && r2 == 42` with no assertion failure. The reason is straightforward: regardless of the choice made for the read of `y` in thread 1, a write to `x` of value 42 is always made. The machinery of MRD recognises this, lifting as it should.

§ 5. Conclusion

MRD is one possible solution to the thin-air problem. Its machinery is *orthogonal* to the conditions of the existing memory model, and as a consequence, it could be implemented within the standard in a localised and self-contained way. Moreover, all of the refinements made to the current standard would be carried over. MRDer makes it clear that MRD can be evaluated efficiently, and enables us to quickly answer questions about how it behaves.

Refinement of MRD will benefit from:

- interesting tests or optimisations that should be supported in the memory model, and
- discussion of the details with members of SG1.

We believe MRDC hits quite a few desirable properties for a revision of the C++ concurrency model:

- continued support of `ld;st` re-ordering,
- reasoning about programs with RFUB behaviour,
- implementability at no additional cost on major architectures (ARM, POWER, x86, RISC-V),
- augmentation of RC11, incorporating all known fixes to C++ concurrency,
- extension, rather than re-creation, of the existing standard,
- potential route to redefining `memory_order::consume` using semantic dependency, and
- application to other memory models with an *axiomatic* representation, e.g. Java, Linux and OpenCL.